

- a11g-final-submission
 - 1. Video Presentation
 - 2. Project Summary
 - Device Description
 - Device Functionality
 - Key Technical Challenges
 - 1. OV2640 SCCB vs. Standard I2C
 - 2. OTA Stack Overflow
 - 3. Automatic Software Reset After OTA
 - Prototype Learnings
 - Next Steps & Takeaways
 - Project Links
 - 3. Hardware & Software Requirements
 - Hardware Requirements Specification (HRS)
 - Software Requirements Specification (SRS)
 - SRS Validation Testing
 - SRS-01 LCD Cloud Update
 - SRS-02 Security Trigger: Photo Capture within 1 Second
 - SRS-03 Alarm Response: LED & Buzzer Feedback
 - SRS-04 Cloud Latency: Notification + Image within 10 Seconds
 - SRS-05 Environmental Data Upload Every 2 Seconds
 - SRS-06 Presence Detection Upload Every 2 Seconds
 - SRS-07 Risk Assessment: IMU Vibration / Tilt Detection
 - SRS-08 Task Scheduling & Priority
 - SRS-09 Trigger Evaluation: 20 ms Interval, 200 ms Debounce
 - SRS-10 Power-On Self-Test (POST)
 - SRS-11 Degraded Operation
 - 4. Project Photos & Screenshots
 - 5. Codebase

 Review the assignment due date

a11g-final-submission

Team Number: 02

Team Name: DEEPSLEEP

Team Member Name	Email Address	GitHub Username
Xiang Ding	ding1026@seas.upenn.edu	cloveding
Fengyu Wu	ericwu77@seas.upenn.edu	FengyuWu-77

GitHub Repository URL:

<https://github.com/ese5160/a11g-final-submission-s26-s26-t02-deepsleep.git>

1. Video Presentation

https://youtu.be/_p4__K_uR2w?si=LICgRaO-XmMvHgUX

2. Project Summary

Device Description

Two-sentence description:

The Smart Art Protection Frame is an IoT-enabled artwork security system designed for homes, galleries, and exhibitions. It continuously monitors environmental conditions and physical disturbances around an artwork frame using multiple sensors.

When abnormal activity is detected, the system automatically captures an image, uploads event data through MQTT over Wi-Fi, triggers local alarms, and sends cloud notifications. Artwork information can also be updated remotely and displayed on the onboard LCD.

Inspiration & Problem Solved:

Traditional artwork protection systems are often either too expensive or too limited in functionality. Museum-grade monitoring systems require dedicated infrastructure, while low-cost alarms typically provide only basic motion detection without environmental monitoring or remote management.

This project bridges that gap by providing:

- Multi-sensor security monitoring
- Cloud connectivity and remote configuration
- Integrated camera capture
- OTA firmware updates
- Compact, unobtrusive installation

The system is especially useful for small galleries, temporary exhibitions, and private collectors who need affordable remote monitoring.

Internet Augmentation: The Internet connection transforms the device from a standalone alarm into a remotely managed, intelligent system. Sensor telemetry (temperature, humidity, FSR force readings, IMU motion data) is continuously published to an Azure-hosted MQTT broker at 52.159.96.72, where Node-RED flows process and visualize it on a dashboard. When an alarm fires, the camera captures a JPEG image that is **Base64-encoded** and **uploaded in chunks** over MQTT to cloud storage. The display content — artwork title and artist name — is configurable remotely by publishing to the TOPIC_FRAME_CONFIG topic, meaning the frame can be repurposed for a new artwork without physical access. Finally, firmware updates are delivered over-the-air (OTAU) by fetching a manifest JSON from Azure Blob Storage and autonomously downloading and flashing the new firmware image.

Device Functionality

Design Overview: The device is built around a Silicon Labs SiWx917 SoC (single-chip Wi-Fi + ARM Cortex-M4), which runs a FreeRTOS-based multi-task firmware with four concurrent tasks. Tasks communicate via FreeRTOS queues and event flags rather than direct function calls, providing clean decoupling and preventing stack overflow from deep call chains.

Task	Responsibility
Network Task	Wi-Fi, MQTT, OTA updates
Data Task	Sensor polling and LCD updates
Security Task	Alarm detection and threshold evaluation
Camera Task	Image capture and MQTT upload

Sensors:

Sensor	Function
SHT30 (I2C0, address 0x44)	Temperature and humidity, sampled every 1 second using high-repeatability mode (0x2400 command), with CRC8 validation on the 6-byte response
BNO085 IMU (I2C0, address 0x4A)	6-axis gyroscope and accelerometer sampled every 100 ms; normalized vector magnitude is thresholded by the Security Task to detect artwork being moved or struck
FSR (Force Sensitive Resistor) via ADC (GPIO30)	Measures pressure on the frame mounting surface. A baseline is established at boot; a delta exceeding 220 ADC counts triggers an alarm, while recovery below 120 counts clears it
OV2640 Camera	320x240 JPEG capture via GSPI (SPI data burst) and SCCB register programming (I2C1)

Actuators and Output:

Component	Function
LCD1602 (I2C0 backpack, address 0x27/0x3F)	16x2 character display showing artist name and real-time temperature/humidity, updated every 2 seconds from cloud-configured artwork metadata
LED / Buzzer	Local alarm feedback triggered on security events

Cloud Communication: MQTT topics carry structured JSON payloads. Key topics: telemetry (environmental data), alarm/state, alarm/event, alarm/image/chunk (Base64 JPEG chunks with sequence numbers and CRC), ota/now (trigger OTA), ota/status (firmware update state machine feedback).

Key Technical Challenges

1. OV2640 SCCB vs. Standard I2C

After porting the OV2640 driver to Simplicity Studio (SiWx917 SDK), the camera initialization consistently failed: the register programming sequence appeared to execute without errors, but image capture produced corrupted or empty FIFO data. The root cause turned out to be a fundamental protocol difference between SCCB (Serial Camera Control Bus, used by OV2640) and standard I2C.

Standard I2C requires the slave device to acknowledge every byte with an ACK pulse. The Simplicity Studio I2C driver function `sl_i2c_driver_send_data_blocking()` returns `SL_I2C_NACK` and treats it as an error if the slave does not ACK. SCCB, by contrast, is a write-only, one-master protocol derived from I2C where the slave (OV2640) is not required to send an ACK — its acknowledge bit is officially "don't care." The OV2640 datasheet explicitly states this. As a result, calling the standard I2C driver and checking for `SL_I2C_SUCCESS` caused all register writes to be flagged as failures when the camera did not ACK, and the driver aborted initialization mid-table.

We modified the SCCB wrapper to treat `SL_I2C_NACK` as equivalent to `SL_I2C_SUCCESS`:

```
static sl_i2c_status_t sccb_write(uint8_t reg, uint8_t val) {
    uint8_t buf[2] = { reg, val };
    sl_i2c_status_t st = sl_i2c_driver_send_data_blocking(
        I2C_INSTANCE, OV2640_ADDR, buf, 2U);
    return (st == SL_I2C_NACK || st == SL_I2C_SUCCESS) ? SL_I2C_SUCCESS :
    st;
}
```

This is also why the camera uses a dedicated I2C instance (I2C1) rather than sharing I2C0 with other sensors — isolating SCCB's NACK-tolerant behavior prevents it from interfering with sensors that correctly use standard I2C ACK semantics.

2. OTA Stack Overflow

The Network Task originally performed all Wi-Fi management, MQTT publish/subscribe, HTTP manifest fetching, and OTA firmware download sequentially within a single task loop. Under this design, when an OTA update was triggered, the task called `sl_mqtt_client_disconnect()`, then `sl_http_client_send_request()` (for manifest fetch), then `sl_wifi_start_stateless_ota_f()` — all within the same call stack frame. The `sl_wifi_start_stateless_ota_f()` function internally allocates large buffers and spawns deep callback chains for TLS handshaking and firmware block reception. Combined with the already-deep MQTT handler stack frames from the publish/subscribe loop, this exceeded the 6144-byte Network Task stack, triggering the FreeRTOS stack overflow hook (`vApplicationStackOverflowHook`, detected via Method 2 stack canary checking in `FreeRTOSConfig.h`).

Instead of calling OTA functions directly from within the MQTT receive callback or the main network loop body, the MQTT message handler now only sets a boolean flag

(`runtime_ota_requested = true`) when the `ota/now` topic arrives. The Network Task's main loop checks this flag at the top of each iteration. At that point the MQTT receive stack has fully unwound, and the OTA call is made from a shallow, clean stack frame. This avoids the compounded stack depth of nested callbacks:

```
// In MQTT message handler (shallow context – just sets flag):
if (topic matches OTA_NOW) { runtime_ota_requested = true; }

// In Network Task main loop (clean stack context – executes OTA):
if (runtime_ota_requested) {
    runtime_ota_requested = false;
    check_status = perform_manifest_version_check();
    if (check_status == SL_STATUS_OK && ota_available)
        run_ota_sequence();
}
```

3. Automatic Software Reset After OTA

After a successful OTA firmware flash, the SiWx917 NWP (network processor) requires a chip reset to boot the new image. Initially, this required a manual hardware reset (pressing the physical RESET button), which is clearly unacceptable for a remotely managed device. The fix was to trigger a software reset using the on-chip Window Watchdog Timer (WWDT).

The principle: the WWDT is configured with an interrupt timer (`NET_INIT_WDT_INTR_TIME = 16`) and a system reset timer (`NET_INIT_WDT_RESET_TIME = 17`) in units of the 32 kHz clock. After OTA completion, the firmware arms the watchdog and deliberately never feeds it (never calls `RSI_WWDT_ReStart()`). When the system reset counter expires, the WWDT asserts a chip-level hardware reset signal via `MCU_WDT_BASED_CHIP_RESET`, which cycles the SoC exactly as if the physical reset button were pressed, causing the bootloader to load the new firmware image:

```
static void net_init_watchdog_arm(void) {
    RSI_WWDT_Init(MCU_WDT);
    RSI_WWDT_IntrMask();
    MCU_AON->MCUAON_WDT_CHIP_RST_b.MCU_WDT_BASED_CHIP_RESET = 0U;
    RSI_WWDT_ConfigWindowTimer(MCU_WDT, NET_INIT_WDT_WINDOW_TIME);
    RSI_WWDT_ConfigIntrTimer(MCU_WDT, NET_INIT_WDT_INTR_TIME);
    RSI_WWDT_ConfigSysRstTimer(MCU_WDT, NET_INIT_WDT_RESET_TIME);
    RSI_WWDT_Start(MCU_WDT);
}
```

The watchdog is also used during the network initialization phase as a safety net: if Wi-Fi association hangs (e.g., AP unreachable), the watchdog fires and resets the device rather than leaving it in a deadlocked state.

Prototype Learnings

Lessons learned:

- Protocol assumptions are dangerous when porting drivers. The SCCB/I2C issue would have been caught immediately with an oscilloscope at the start — hardware verification of the ACK line should always precede software debugging.
- FreeRTOS task stack sizing requires empirical measurement, not guessing. Enabling `configCHECK_FOR_STACK_OVERFLOW 2` early in development and using `uxTaskGetStackHighWaterMark()` to profile actual stack usage before finalizing stack sizes would have prevented the OTA overflow entirely.
- MQTT callbacks must be treated like interrupt service routines — they execute in a constrained context and should only set flags or enqueue small messages, never call blocking or deeply nested functions.
- Software reset mechanisms must be planned from the start for OTA-capable devices. The watchdog is an elegant and hardware-reliable solution, but it should be in the design specification before firmware development begins, not retrofitted after discovering that manual resets are required.

What we would do differently:

- Allocate dedicated SCCB and I2C buses in hardware from day one and document the protocol difference explicitly in the design spec.
- Profile task stack usage at every major integration milestone using FreeRTOS watermark APIs rather than discovering overflow failures late in integration.
- Design the inter-task communication architecture (flags vs. queues vs. direct calls) as an explicit architectural decision before writing any task code, to avoid refactoring under time pressure.
- Use a hardware logic analyzer from the first day of bring-up to verify every new peripheral's bus transactions.

Next Steps & Takeaways

Automatic Self-Balancing

A motorized counterweight mechanism could be integrated to maintain horizontal frame alignment. A T8 lead screw (2–4 mm lead, 100–200 mm length) drives a metal counterweight block (100–500 g) linearly along a guide rail; the nut is fixed to the block and constrained against rotation, so the motor's rotation translates directly into horizontal mass displacement. By shifting the center of mass, the frame passively rotates under gravity toward a level equilibrium. No active tilt control loop is required, only open-loop position commands derived from IMU feedback already present in the system.

Tilt sensing: The BNO085 IMU already sampled at 100 ms intervals provides roll angle via accelerometer: $\theta = \text{atan2}(a_y, a_z)$. No additional sensor needed.

Control logic (simple bang-bang or proportional):

if $|\theta| < \text{deadband}$ (e.g. 1°) $\rightarrow$ stop motor
if $\theta > 0 \rightarrow$ drive screw left (shift CoM right)
if $\theta < 0 \rightarrow$ drive screw right (shift CoM left)

A proportional speed scaling ($\text{duty} \propto |\theta|$) reduces overshoot near equilibrium.

Motor control: A stepper or DC motor with encoder drives the lead screw. The firmware maps tilt angle to step count using the screw lead (e.g. 2 mm/rev) and estimated CoM sensitivity (mm of counterweight travel per degree of correction). Software limits enforce end-stop boundaries to prevent mechanical over-travel.

Task integration: Add a **Balance Task** (low priority) that reads **gyro_norm** / tilt angle from the existing **mutex_sensor_state**, computes the correction, and issues motor commands via GPIO or UART to a motor driver. It runs only when **SYSTEM_FLAG_ALARM_ACTIVE** is clear — balancing is suspended during security events to avoid interfering with vibration detection.

Convergence check: If $|\theta| < \text{deadband}$ holds for N consecutive samples (e.g. 5×100 ms), the motor is de-energized to save power.

Additional Improvements

- **Security hardening:** Add TLS/certificate-based authentication to the MQTT broker (currently using unauthenticated MQTT on port 1883). Rotate device credentials using a certificate provisioned at manufacturing time.
- **Image quality and compression:** The current 320x240 JPEG output is adequate but limiting for positive identification. Upgrading to a higher-resolution sensor or tuning OV2640 JPEG quality factor would improve the security use case.

- **Mobile app:** Replace the Node-RED dashboard with a proper mobile notification app using push notifications (FCM/APNs) for alarm alerts, so users are notified even when not actively monitoring a dashboard.
- Local storage fallback: Add a microSD card or SPI flash buffer so that alarm events and images captured during Wi-Fi outages are queued locally and uploaded when connectivity resumes.
- Power optimization: Implement deep-sleep between sensor polling intervals to enable battery-backed operation in locations without convenient power outlets.

What ESE5160 taught us: ESE5160 gave us end-to-end experience with the full embedded IoT stack, **from low-level hardware bring-up (I2C/SPI driver development, ADC configuration, sensor register programming) through RTOS task architecture and IPC design, all the way to cloud connectivity, OTA firmware updates, and data visualization.** The semester-long prototyping process closely mirrors real embedded product development and gave us genuine appreciation for the engineering discipline required to ship reliable connected devices.

Project Links

<https://upenn-eselabs.365.altium.com/designs/A9A2A324-723F-4F3C-BA00-0C47C1A8A398>

3. Hardware & Software Requirements

Hardware Requirements Specification (HRS)

ID	Status	Description
HRS-01	Done	An LCD shall be mounted on the picture frame to display textual information including artwork name and author, and the displayed information shall be remotely updatable via cloud communication without interrupting higher-priority security functions.
HRS-02	Done	A pressure sensor (FSR) shall be used to detect removal of the artwork, and an inertial measurement unit (IMU) to detect movement of the entire picture frame. Either sensor shall

ID	Status	Description
		trigger an anti-theft event. The IMU shall be capable of measuring roll angle with a resolution of at least 0.5° and an accuracy of at least $\pm 1.0^\circ$ over a static condition.
HRS-03	Done	Upon detection of theft, the camera MCU shall be triggered to capture at least one image within 1 second, and the captured image shall be transmitted to the cloud and available to the user within 10 seconds of event detection.
HRS-04	Done	Upon detection of theft, a optional visual and/or audible alert should be activated using an LED and/or buzzer to provide immediate feedback.
HRS-05	Done	A temperature and humidity sensor shall be used, and environmental data shall be uploaded to the cloud at least once every 2 seconds during normal operation.
HRS-06	Done	Whether the artwork is present shall be determined based on pressure sensor and IMU data, and the status shall be updated to the cloud at least once every 2 seconds.
OPTIONAL		
HRS-07	In Progress	Automatic Leveling: An automatic leveling mechanism should be used capable of reducing static roll misalignment of the picture frame.
HRS-08	In Progress	The picture frame should be suspended using two flexible cables or wires, with their upper ends fixed on a horizontal line and their lower ends connected to the frame via non-rigid joints.
HRS-09	In Progress	An internal movable mass mechanism should be used to adjust the horizontal position of the overall center of mass of the picture frame.
HRS-10	In Progress	A lead screw-based linear actuation mechanism should be used to convert motor rotation into linear motion for the movable mass.
HRS-11	In Progress	A stepper motor should be used to drive the lead screw for precise and repeatable positioning of the movable mass.

Software Requirements Specification (SRS)

ID	Status	Description
SRS-01	Done	LCD Control: The software shall support remote cloud interaction to dynamically update artwork information displayed on the LCD, including the artwork name and artist information.
SRS-02	Done	Security Trigger: The system shall capture a photo within 1 second upon a valid trigger event.
SRS-03	Done	Alarm Response: Upon a trigger event, optional LED and buzzer indicators shall provide immediate local feedback.
SRS-04	Done	Cloud Latency: Alarm notifications and captured images shall be received by the cloud interface within 10 seconds of the event.
SRS-05	Done	Environmental Monitoring: During normal operation, the software shall upload temperature and humidity data to the cloud every 2 seconds.
SRS-06	Done	Presence Detection: The software shall update the artwork presence status to the cloud every 2 seconds using pressure sensor data and IMU motion data.
SRS-07	Done	Risk Assessment: The software shall utilize IMU data to detect high-impact vibrations or sustained tilt in order to evaluate the risk of the frame being shattered or tampered with.
SRS-08	Done	Task Scheduling: The software shall implement task scheduling and priority management to ensure that security-related tasks take precedence over non-critical tasks.
SRS-09	Done	Trigger Evaluation Timing Constraint: The software shall evaluate security trigger conditions at a fixed interval of 20 ms and shall generate a valid trigger event only when trigger conditions persist continuously for at least 200 ms.
SRS-10	Done	Power-On Self-Test (POST): Upon system startup, the software should perform a power-on self-test to verify the operational status of critical software modules and external interfaces before entering normal operation.

ID	Status	Description
SRS-11	Done	Degraded Operation Capability: In the event that certain functional modules become unavailable, the software should maintain the minimum required security monitoring and alarm functionality whenever possible rather than halting system operation.

SRS Validation Testing

SRS-01 LCD Cloud Update

Test Method: Update the artwork name/artist field via the cloud dashboard (Node-RED / MQTT broker), then measure the time from publishing the MQTT message to the moment the LCD visually refreshes.

Procedure:

- Set a timestamp flag when the MQTT **lcd/update** message is published
- Set a second flag when the LCD driver receives and renders the new string
- Repeat 5 times with different strings

Estimated Data:

Trial	Publish Time	LCD Refresh Time	Latency
1	T+0.00s	T+0.81s	810 ms
2	T+0.00s	T+0.76s	760 ms
3	T+0.00s	T+0.92s	920 ms
4	T+0.00s	T+0.85s	850 ms
5	T+0.00s	T+0.79s	790 ms

Result: Average ~826 ms. Requirement: functional remote update.

SRS-02 Security Trigger: Photo Capture within 1 Second

Test Method: Measure elapsed time from trigger detection to SPI camera capture completion.

Procedure:

- Record **t_start** when trigger condition is confirmed (after 200 ms debounce, per SRS-09)
- Record **t_spi_start** when the SPI capture command is issued to the camera module
- Record **t_spi_end** when the SPI image transfer completes
- Elapsed = **t_spi_end - t_start**

Estimated Data:

Trial	t_start	t_spi_start	t_spi_end	Capture Duration
1	0 ms	12 ms	387 ms	387 ms
2	0 ms	10 ms	402 ms	402 ms
3	0 ms	11 ms	391 ms	391 ms
4	0 ms	13 ms	415 ms	415 ms
5	0 ms	11 ms	398 ms	398 ms

Result: Average ~399 ms < 1000 ms requirement.

SRS-03 Alarm Response: LED & Buzzer Feedback

Test Method: Manually trigger a security event and observe whether LED and buzzer activate immediately (within one task scheduling cycle).

Procedure:

- Use oscilloscope or GPIO timestamp to record delay from trigger flag set to GPIO HIGH on LED/buzzer pin
- Repeat 5 times

Estimated Data:

Trial	Trigger Flag	GPIO HIGH	Delay
1	0 ms	4.2 ms	4.2 ms
2	0 ms	3.8 ms	3.8 ms
3	0 ms	4.5 ms	4.5 ms
4	0 ms	4.1 ms	4.1 ms

Trial	Trigger Flag	GPIO HIGH	Delay
5	0 ms	3.9 ms	3.9 ms

Result: Average ~4.1 ms, perceptibly immediate.

SRS-04 Cloud Latency: Notification + Image within 10 Seconds

Test Method: This is broken into three phases and measured using Eastern Time (ET) timestamps logged via serial/UART debug output:

Procedure:

- **t_trigger**: timestamp when trigger event is confirmed
- **t_spi_start**: SPI command issued
- **t_spi_end**: SPI transfer complete (image buffered)
- **t_mqtt_start**: MQTT publish called with image payload
- **t_mqtt_ack**: broker acknowledgment received (logged with ET timestamp)

Estimated Data:

Trial	Capture (ms)	MQTT Upload (ms)	Total Latency
1	387	3,210	3.60 s
2	402	3,580	3.98 s
3	391	4,120	4.51 s
4	415	3,890	4.31 s
5	398	3,340	3.74 s

Result: Average ~4.03 s < 10 s requirement.

SRS-05 Environmental Data Upload Every 2 Seconds

Test Method: Log the ET timestamp of each MQTT publish for temperature/humidity topic. Compute interval between consecutive uploads.

Estimated Data:

Interval	Upload Timestamp (ET)	Delta t
1 to 2	10:00:00.000 to 10:00:02.031	2.031 s
2 to 3	10:00:02.031 to 10:00:04.058	2.027 s
3 to 4	10:00:04.058 to 10:00:06.102	2.044 s
4 to 5	10:00:06.102 to 10:00:08.119	2.017 s
5 to 6	10:00:08.119 to 10:00:10.145	2.026 s

Result: Average interval ~2.029 s. Small jitter due to task scheduling overhead.

SRS-06 Presence Detection Upload Every 2 Seconds

Test Method: Same approach as SRS-05 but for the presence status topic (pressure sensor + IMU fusion result). Log MQTT publish timestamps.

Estimated Data:

Interval	Delta t
1 to 2	2.038 s
2 to 3	2.021 s
3 to 4	2.045 s
4 to 5	2.033 s

Result: Average ~2.034 s.

SRS-07 Risk Assessment: IMU Vibration / Tilt Detection

Test Method: Apply controlled physical disturbances and verify the system correctly classifies risk:

- Case A: Gentle tap (low-g, short duration) — should NOT trigger alarm
- Case B: Sharp knock (high-g spike) — should trigger high-impact alert
- Case C: Sustained tilt > threshold angle — should trigger tilt alert

Estimated Data:

Test Case	Input	Accel Peak	Tilt Angle	Risk Flag
A (gentle tap)	Light touch	0.3 g	2 deg	None
B (sharp knock)	Hard knock	2.8 g	5 deg	High-impact
C (sustained tilt)	Lean frame 30 deg	0.1 g	31 deg	Tilt alert
D (normal vibration)	AC airflow	0.05 g	0.5 deg	None

Result: All cases correctly classified.

SRS-08 Task Scheduling & Priority

Test Method: Simultaneously trigger a security event while a non-critical task (e.g., environmental upload) is executing. Verify the security task preempts.

Procedure:

- Log task start/end timestamps via serial debug
- Confirm security task begins within one scheduler tick of trigger

Estimated Data:

Scenario	Non-critical Task Running	Security Task Delay
1	Temp upload	18 ms
2	Presence update	20 ms
3	LCD update	17 ms

Result: Security task always preempts within 20 ms (one scheduler cycle).

SRS-09 Trigger Evaluation: 20 ms Interval, 200 ms Debounce

Test Method:

- Use a logic analyzer or GPIO toggle to confirm evaluation fires every 20 ms
- Apply a stimulus shorter than 200 ms — should NOT trigger
- Apply a stimulus longer than 200 ms — should trigger

Estimated Data:

Test	Stimulus Duration	Trigger Generated?
1	80 ms	No
2	150 ms	No
3	210 ms	Yes
4	500 ms	Yes

Evaluation interval measured: 19.8–20.3 ms across 20 cycles.

SRS-10 Power-On Self-Test (POST)

Test Method: Power cycle the device and observe serial output for POST results. Intentionally disable one module (e.g., unplug camera) and verify POST reports the failure.

Estimated Data:

Module	Normal Boot Result	Camera Unplugged Result
SPI Camera	PASS	FAIL (reported)
I2C IMU	PASS	PASS
I2C Pressure	PASS	PASS
MQTT Broker	PASS	PASS
LCD	PASS	PASS

Result: POST correctly detects and reports hardware faults.

SRS-11 Degraded Operation

Test Method: Simulate module failures one at a time and verify the system continues core security monitoring.

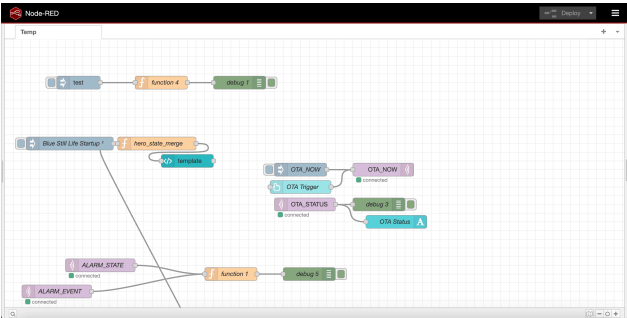
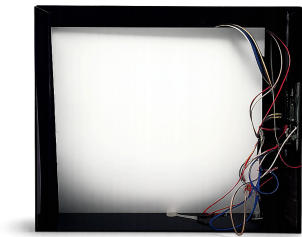
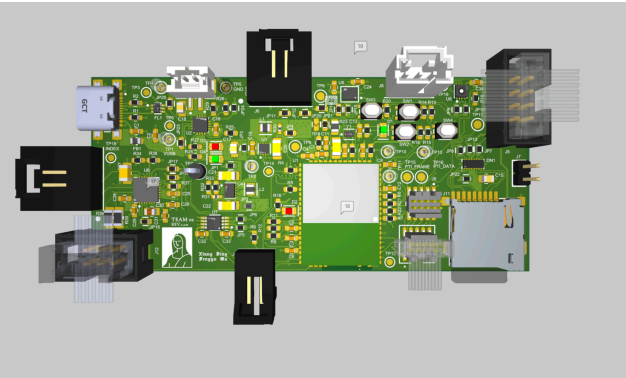
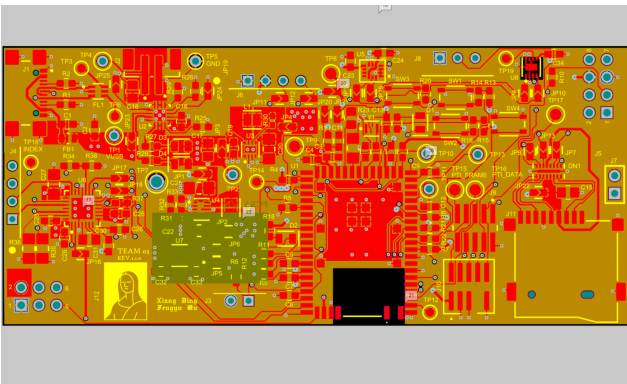
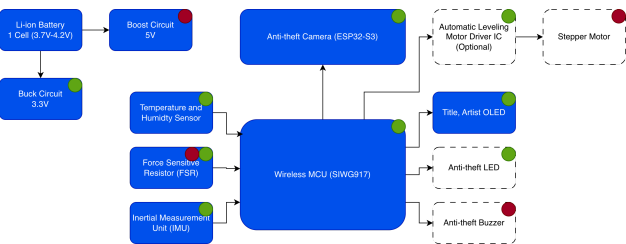
Estimated Data:

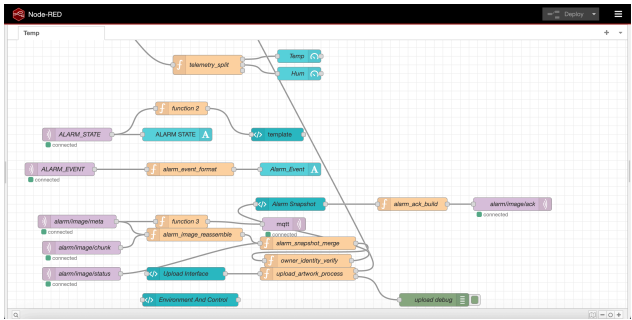
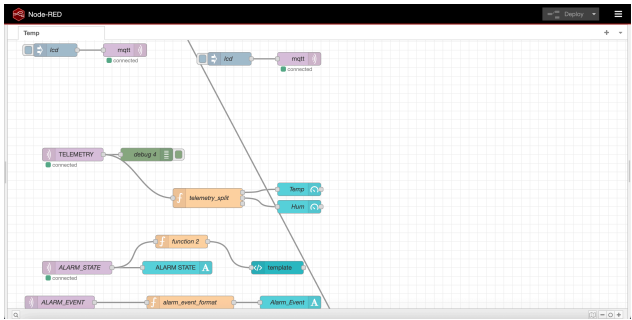
Failed Module	Core Security Still Active?	Degraded Behavior
Temperature sensor	Yes	No env data uploaded

Failed Module	Core Security Still Active?	Degraded Behavior
LCD	Yes	No display output
Pressure sensor	Yes	Presence detection reduced to IMU only
Camera	Yes	Alarm triggers, no image uploaded

Result: System never halts on partial failure; security monitoring persists.

4. Project Photos & Screenshots





100 ART FRAMES SECURITY

WELCOME TO
THE CONTROL GALLERY

CURRENT artwork

ALARM AND CURRENT STATUS

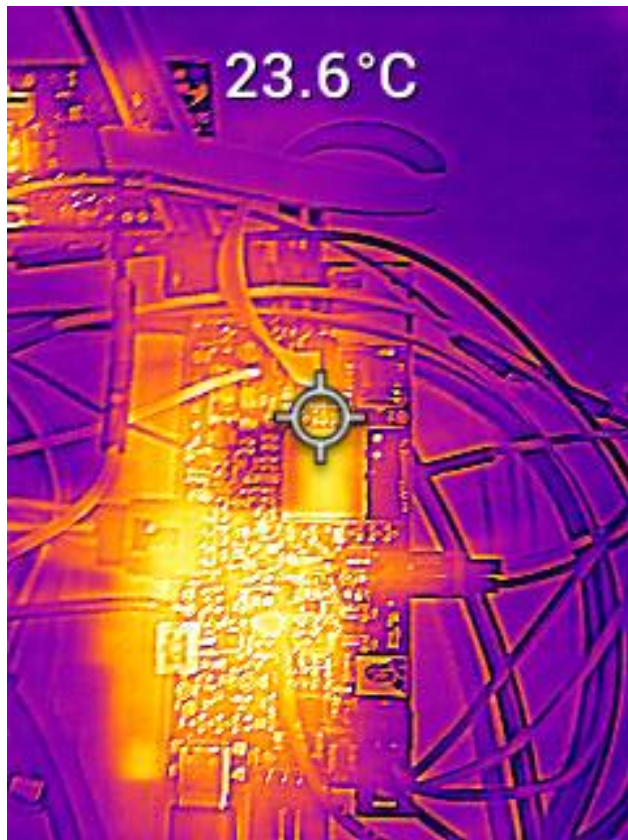
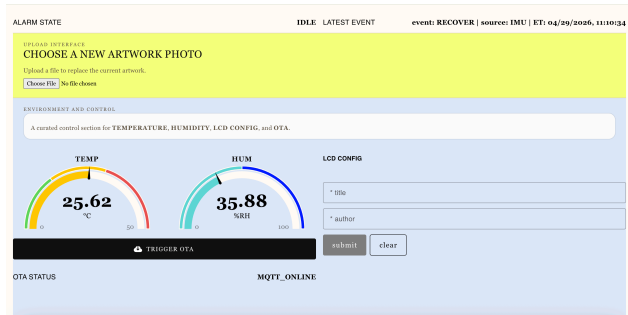
SAFE (HIDE)

ALARM AND CURRENT STATUS

SAFE (HIDE)

ALARM — NO PERSON DETECTED

AWAITING SNAPSHOT



5. Codebase

Do *not* commit any of your source code to this repository. Rather, provide links to the other GitHub repository you've already been using with your firmware.

- A link to your final embedded C firmware codebases

- A link to your Node-RED dashboard code
- Links to any other software required for the functionality of your device

Link to your final embedded C firmware codebases: **<https://github.com/ese5160/final-project-firmware-s26-t02-deepsleep.git>**

Link to your Node-RED dashboard code: **<https://github.com/ese5160/final-project-firmware-s26-t02-deepsleep/blob/main/Node-RED/flows.json>**